Serial Python Minimalis: Tutorial #10-Plot Grafik

Ahmad R. T. Nugraha

ver. 21 April 2026

Matplotlib adalah pustaka (*library*) pembuatan/*plotting* grafik yang paling populer untuk Python. Pustaka ini dapat digunakan untuk menghasilkan visualisasi data secara cepat. Dengan ketelitian, grafik berkualitas tinggi dan siap publikasi dalam berbagai format pun dapat dihasilkan. Sesi kali ini difokuskan pada fungsionalitas `pyplot` dari Matplotlib, yang digunakan untuk membuat dan menganotasi (menulis berbagai label) pada grafik ilmiah menggunakan antarmuka prosedural yang sederhana. `pyplot` perlu diimpor, dan biasanya diberi alias menggunakan pengenal yang disingkat, lazimnya adalah `plt`:

```
import matplotlib.pyplot as plt
```

1 Plot Garis

Penggunaan sederhana untuk memplot dan memberi label pada beberapa titik data adalah dengan memanggil `plt.plot` bersama dua *list* ataupun *array*/larik (sangat disarankan langsung menggunakan larik NumPy) berisi angka sebagai berikut:

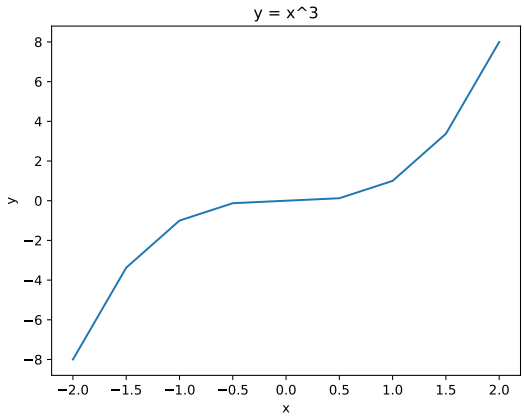
```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([-2, -1.5, -1, -0.5, 0, 0.5, 1, 1.5, 2])
y = x**3

plt.plot(x, y)
```

Di dalam skrip Python atau sesi Python interaktif, biasanya tidak akan ada yang muncul di layar sampai kita memanggil `plt.show()`. Jadi, sebelum memanggil `plt.show()`, berbagai anotasi dapat ditambahkan, seperti label untuk sumbu dan sebuah judul:

```
plt.xlabel('x')
plt.ylabel('y')
```

```
plt.title('y = x^3')
plt.show()
```

Sementara itu, dalam Jupyter Notebook, grafik akan langsung ditampilkan segera setelah sel yang berisi pernyataan `plt.plot()` dieksekusi, sehingga kelengkapan anotasi di atas harus berada di dalam sel yang sama dengan `plt.plot()`. Hasil eksekusi perintah-perintah dasar Matplotlib tersebut ditunjukkan pada Gambar 1.



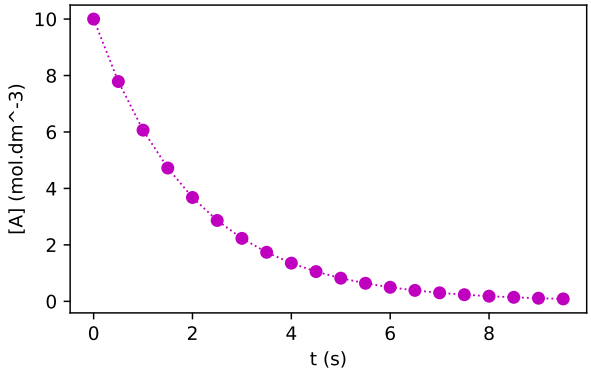
Gambar 1: Pemanfaatan Matplotlib untuk plot garis $y = x^3$ dengan menggunakan sembilan titik.

Walau tampaknya "cukup" untuk keperluan melihat bentuk grafik tertentu, tampilan pada Gambar 1 masih **belum memenuhi standar kelaziman grafik ilmiah** untuk publikasi. Secara bertahap, kita akan belajar kustomisasi dan pendetailan grafik ilmiah dengan Matplotlib. Sebagai awalan, grafik dapat dikustomisasi dengan penanda (`marker`), warna (*color*, `c`), ketebalan (*linewidth*, `lw`), serta gaya garis (*linestyle*, `ls`) menggunakan argumen-argumen tambahan, sebagaimana diperinci pada Tabel 1– 3. Sebagai contoh, kita buat gambar baru untuk plot grafik konsentrasi suatu materi, $[A] = [A]_0 e^{-kt}$, yang ditunjukkan pada Gambar 2.

```
import numpy as np
import matplotlib.pyplot as plt

A0, k = 10, 0.5
t = np.arange(0, 10.1, 0.5)
A = A0 * np.exp(-k * t)

plt.figure(figsize=(5,3))
plt.plot(t, A, c='m', lw=1, ls=':', marker='o')
plt.xlabel('t (s)')
plt.ylabel('[A] (mol.dm^-3)')
```

```
plt.show()
```

Perhatikan bahwa kita dapat memanfaatkan fungsi `plt.figure()` dengan argumen `figsize=(width,height)` untuk spesifikasi lebih terperinci terkait ukuran gambar. Parameter `width` dan `height` dinyatakan dalam satuan inci.



Gambar 2: Plot $[A] = [A]_0 e^{-kt}$.

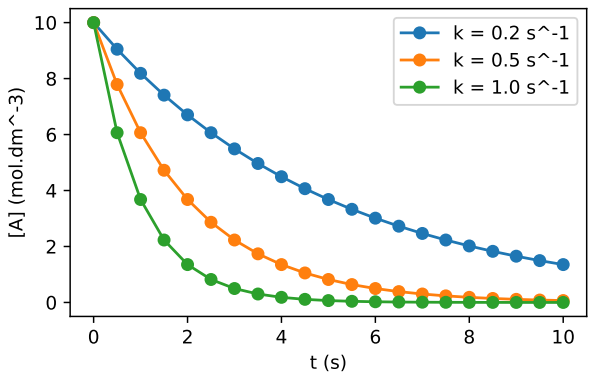
Tabel 1: Kode warna Matplotlib. Ada versi kode warna dasar satu huruf yang dapat dispesifikasi sebagai argumen atau parameter saat pemanggilan `plt.plot()`, serta ada siklus urutan warna bawaan palet "Tableau 10".

Kode warna dasar	Warna Tableau
`b` = biru (*blue*)	`tab:blue`
`g` = hijau (*green*)	`tab:orange`
`r` = merah (*red*)	`tab:green`
`c` = sian (*cyan*)	`tab:red`
`m` = magenta (*magenta*)	`tab:purple`
`y` = kuning (*yellow*)	`tab:brown`
`k` = hitam (*black*)	`tab:pink`
`w` = putih (*white*)	`tab:gray`
	`tab:olive`
	`tab:cyan`

Pemanggilan `plt.plot` secara berulang dengan data yang berbeda akan menambahkan garis-garis baru ke dalam grafik. Dengan adanya beberapa garis yang diplot, ada baiknya kita memberikan perbedaan melalui `string` yang sesuai ke argumen `label`. Dalam kasus seperti Gambar 2, kita dapat membuat beberapa garis dengan nilai k yang berbeda. Sebagai pembedanya, `plt.legend()` harus dipanggil sebelum menampilkan grafik-grafik tersebut.

Tabel 2: Beberapa gaya penanda (*marker*) Matplotlib dengan kode string satu karakter.

Kode	Penanda	Deskripsi
`.`	$\cdot$	Titik (*Point*)
`o`	$\circ$	Lingkaran (*Circle*)
`+`	$+$	Plus (*Plus*)
`x`	$\times$	Silang (*Cross*)
`D`	$\diamond$	Wajik (*Diamond*)
`v`	$\triangledown$	Segitiga menghadap bawah (*Downward triangle*)
`^`	$\triangle$	Segitiga menghadap atas (*Upward triangle*)
`s`	$\square$	Persegi (*Square*)
`*`	$\star$	Bintang (*Star*)



Gambar 3: Plot $[A] = [A]_0 e^{-kt}$ untuk konstanta laju k yang berbeda-beda.

```
import numpy as np
import matplotlib.pyplot as plt

A0 = 10
t = np.arange(0, 10.1, 0.5)
rate_constants = [0.2, 0.5, 1.0]

plt.figure(figsize=(5,3))

for k in rate_constants:
    A = A0 * np.exp(-k * t)
    plt.plot(t, A, marker='o', label=f'k = {k:.1f} s^-1')

plt.xlabel('t (s)')
plt.ylabel('[A] (mol.dm^-3)')
plt.legend()
```

```
plt.show()
```

Hasilnya ditunjukkan pada Gambar 3.

Perhatikan bahwa pada dasarnya kita tidak perlu menentukan warna baru untuk setiap pemanggilan `plt.plot` karena Matplotlib dapat memutar (*cycle*) urutan warna bawaan yang telah ditentukan sebelumnya untuk membedakan garis-garis yang diplot. Namun, untuk tampilan yang lebih baik dalam grafik ilmiah kita tentunya dapat spesifikasi ulang warna-warna setiap garis.

Tabel 3: Properti garis dan penanda Matplotlib.

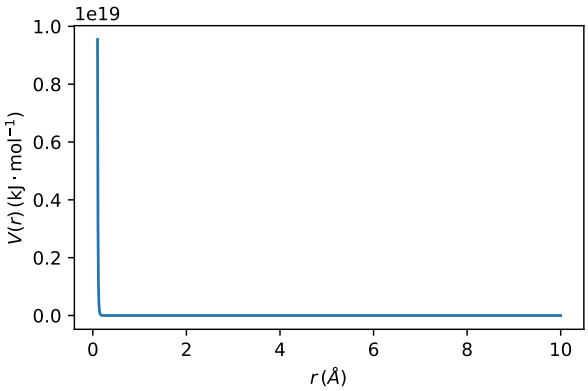
Argumen	Singkatan	Deskripsi
`color`	`c`	Warna garis
`alpha`		Opasitas garis: 0 (sepenuhnya transparan) dan 1 (sepenuhnya buram)
`linestyle`	`ls`	Gaya garis: `'solid'`, `'dotted'`, `'dashed'`, `'dashdot'`; atau disingkat `'-'`, `':'`, `'-'`, `'-.'`
`linewidth`	`lw`	Ketebalan garis dalam poin, nilai bawaannya adalah 1.5 pt.
`markersize`	`ms`	Ukuran penanda, dalam poin
`markevery`		Ditetapkan ke bilangan bulat positif, N, untuk mencetak penanda setiap N titik. Nilai bawaannya, yaitu `None`, mencetak penanda untuk setiap titik
`markerfacecolor`	`mfc`	Warna isian penanda
`markeredgecolor`	`mec`	Warna tepi penanda
`markeredgewidth`	`mew`	Ketebalan tepi penanda, dalam poin

Secara bawaan, Matplotlib memilih batas-batas untuk sumbu x dan y sedemikian rupa sehingga dapat menampilkan seluruh data yang diplot. Untuk menetapkan batas-batas ini secara manual, gunakan `plt.xlim` dan `plt.ylim`. Sebagai contoh, mari kita tinjau plot potensial interatomik Lennard-Jones untuk argon,

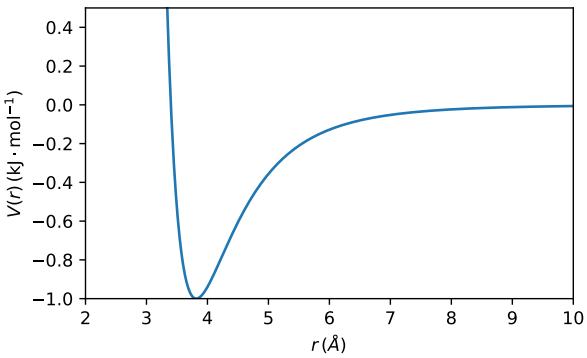
$$V(r) = 4\epsilon \left[\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6} \right]$$

dengan r adalah jarak interatomik, $\epsilon = 3.4$ kJ mol^{-1} adalah kedalaman sumur potensial, dan $\sigma = 3.4$ Å adalah jarak pisah yang potensialnya bernilai nol: $V(\sigma) = 0$. Plot $V(r)$ begitu saja pada suatu kisi (*grid*) berisi nilai-nilai r melalui kode berikut ini tidak akan menghasilkan grafik yang terlalu memuaskan:

```
import numpy as np
import matplotlib.pyplot as plt

rmax = 10
```



Gambar 4: Plot potensial Lennard-Jones untuk argon tanpa menetapkan batas-batas sumbu horizontal maupun vertikal.



Gambar 5: Plot potensial Lennard-Jones untuk argon dengan batas-batas plot yang telah ditetapkan secara lebih baik.

```
r = np.linspace(0.1, rmax, 1000)

# Parameter Lennard-Jones untuk Ar:
# kedalaman sumur (kJ/mol) dan
# jarak pisah yang potensial interatomiknya nol
E, sigma = 1.0, 3.4

def LJ(r, E, sigma):
    fac = (sigma / r)**6
    return 4 * E * (fac**2 - fac)

V = LJ(r, E, sigma)
plt.plot(r, V)
plt.xlabel(r'$r\,(\AA$)')
```

```
plt.ylabel(r'$V(r)\,(\mathrm{kJ} \cdot \mathrm{mol}^{-1})$')
plt.title('Potensial Lennard-Jones untuk Argon')
plt.show()
```

Hasilnya ditunjukkan pada Gambar 4. Di sini, dominasi dari suku tolakan pada potensial (r^{-12}) berujung pada nilai $V(r)$ yang sangat besar pada r kecil, sehingga area sumur tarik-menarik ($V(r) < 0$) tidak terlihat. Solusinya adalah dengan menetapkan batas-batas plot secara tepat:

```
# Mulai sumbu jarak pada 2 Angstrom.
plt.xlim(2, rmax)
# Batasi rentang y antara dasar sumur dan setengah dari besaran
# kedalaman sumur.
plt.ylim(-E, 0.5 * E)
```

Bagian kode di atas dapat ditambahkan di mana pun setelah `plt.plot()` dan sebelum `plt.show()`. Hasilnya ditunjukkan pada Gambar 5. Pada anotasi `plt.xlabel` maupun `plt.ylabel`, kita dapat memanfaatkan format LaTeX dengan menambahkan karakter `r` sebelum tanda petik, kemudian di dalam tanda petik kita perlu mengapit format LaTeX dengan tanda dolar.

2 Plot Sebaran (*Scatter Plots*)

Terdapat beberapa jenis grafik umum lainnya yang disediakan oleh Matplotlib, termasuk plot sebaran (*scatter plot*):

```
plt.scatter(x, y, s=None, c=None, marker=None)
```

Baris kode di atas akan membuat plot sebaran dari data (x, y) menggunakan penanda berukuran `s` dan berwarna `c` yang ditentukan. Nilai bawaan akan digunakan jika tidak ada nilai yang diberikan untuk argumen-argumen tersebut. Jika `s` atau `c` berupa urutan (*sequences*), ukuran dan warna dari penandanya dapat diatur secara spesifik untuk masing-masing titik data.

Untuk menganotasi grafik dengan teks, panggil `plt.text`:

```
plt.text(x, y, s)
```

yang akan menempatkan string `s` pada lokasi (x, y) (dalam koordinat data). Argumen lebih lanjut, `ha` (atau `horizontalalignment`) dan `va` (atau `verticalalignment`) mengontrol bagaimana teks tersebut ditambatkan pada lokasi yang spesifik. Nilai yang valid adalah `ha = 'left'`, `'center'` atau `'right'` dan `va = 'top'`, `'center'`, `'baseline'` atau `'bottom'`. Sebagai contoh, untuk memastikan bahwa titik tengah dari label teks tersebut berada tepat di (x, y), gunakan `ha='center'`, `va='center'`.

Studi Kasus

Kasus 1 Titik Leleh dan Titik Didih Unsur

Berkas `elements_TmTb.csv`, yang dapat diunduh dari situs web `https://scipython.com/chem/ghc`, memuat nilai-nilai titik leleh dan titik didih dari beberapa unsur kimia. Buatlah plot sebaran (*scatter plot*) untuk data ini dengan memberikan warna penanda (*markers*) yang berbeda antara unsur logam dan non-logam. Beri anotasi juga pada grafik tersebut untuk menunjukkan unsur karbon dan unsur yang memiliki selisih terbesar antara titik leleh dan titik didihnya. Untuk kasus ini, unsur-unsur non-logam dapat diasumsikan sebagai unsur-unsur yang terangkum dalam `list` berikut:

```
nonmetals = ['H', 'He', 'C', 'N', 'O', 'F', 'Ne', 'S', 'P',
             'Ar', 'Se', 'Cl', 'Kr', 'Br', 'Xe', 'I', 'Rn']
```

Solusi

File yang disediakan berisi kolom-kolom untuk simbol unsur, titik leleh, dan titik didih yang dipisahkan oleh tanda koma. Jika kita mengintip beberapa baris pertamanya (misalnya menggunakan perintah `!head elements_TmTb.csv` di sistem Linux/WSL), formatnya akan terlihat seperti ini:

```
Element, Tm /K, Tb /K
H, 14.0, 20.3
He, 1.8, 4.2
Li, 453.7, 1603.0
Be, 1560.0, 2742.0
B, 2349.0, 4200.0
C, 3800.0, 4300.0
```

Tantangannya adalah kolom pertama terdiri dari karakter string, bukan angka, sementara kita membutuhkan simbol-simbol unsur tersebut. Kita dapat menggunakan metode `genfromtxt` dari NumPy untuk membaca kolom-kolom ini dan memisahkannya ke dalam larik. Namun, kita perlu menetapkan `dtype=None` (untuk memaksa NumPy menyimpulkan tipe datanya sendiri dan tidak sekadar menganggap kolom pertama sebagai bilangan pecahan) dan juga menentukan pengodean string-nya (biasanya baik ASCII atau UTF-8 sama-sama dapat digunakan).

```python
import numpy as np
import matplotlib.pyplot as plt

# definisikan list nonmetals sesuai soal
nonmetals = ['H', 'He', 'C', 'N', 'O', 'F', 'Ne', 'S', 'P',
             'Ar', 'Se', 'Cl', 'Kr', 'Br', 'Xe', 'I', 'Rn']

# Baca data dari file CSV
elements, Tm, Tb = np.genfromtxt('elements_TmTb.csv', delimiter=',',
                                 skip_header=1, unpack=True,
                                 dtype=None, encoding='utf8')

print(elements[:5])
print(Tm[:5])
print(Tb[:5])
```

Output:

```
['H' 'He' 'Li' 'Be' 'B']
[  14.     1.8  453.7 1560.  2349. ]
[  20.3    4.2 1603.  2742.  4200. ]
```

Kita sebenarnya bisa memanggil `plt.scatter` dua kali (sekali untuk logam dan sekali untuk non-logam) atau merancang sebuah *list* warna yang akan digunakan. Jika kita memilih pendekatan kedua, ternyata lebih mudah jika kita mengonversi larik `elements` menjadi *list* Python biasa terlebih dahulu:

```python
elements = list(elements)
n = len(elements)

# Unsur logam akan ditandai dengan lingkaran biru ('b').
c = ['b'] * n

for symbol in nonmetals:
    if symbol in elements:
        idx = elements.index(symbol)
        # Unsur non-logam akan ditandai dengan lingkaran merah ('r').
        c[idx] = 'r'
```

Unsur manakah yang memiliki selisih terbesar antara titik leleh dan titik didihnya?

```python
# Catatan: Terdapat entri NaN (Not a Number) untuk unsur-unsur
# yang nilai Tm atau Tb-nya belum diketahui.
idx_max_diff = np.nanargmax(Tb - Tm)
print(elements[idx_max_diff])
```

Output:

```
Np
```

Ternyata unsur tersebut adalah Neptunium. Sekarang kita siap memplot datanya dan menambahkan fungsi untuk menganotasi titik-titik tersebut:

```python
plt.scatter(Tm, Tb, c=c, s=15)
plt.xlabel('Titik Leleh (K)')
plt.ylabel('Titik Didih (K)')

def annotate_with_element_symbol(symbol):
    i = elements.index(symbol)
    # Tambah sedikit offset agar teks tidak menutupi titik penanda
    plt.text(Tm[i] + 50, Tb[i] + 50, symbol, ha='left', va='bottom')

annotate_with_element_symbol('C')
annotate_with_element_symbol('Np')

plt.show()
```

Unsur non-logam ditandai dengan warna merah dan logam dengan warna biru. Anotasi menunjukkan posisi Karbon (C) dan Neptunium (Np).

Kasus 2 Kelarutan Kalium Halida

Data berikut berkaitan dengan kelarutan tiga kalium halida (dalam gram garam per 100 gram air) sebagai fungsi dari suhu:

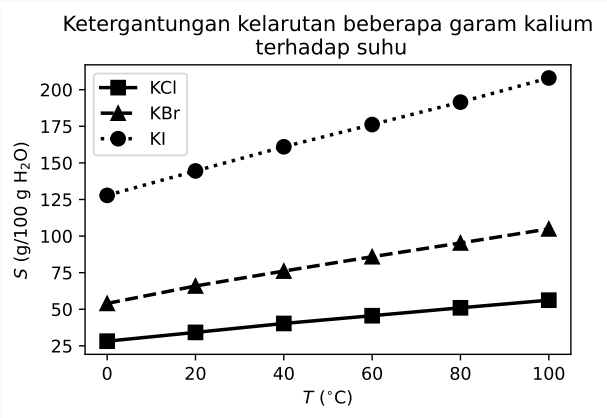
T / °C	KCl	KBr	KI
0	28.2	54.0	127.8
20	34.2	65.9	144.5
40	40.3	76.1	161.0
60	45.6	85.9	176.2
80	51.0	95.3	191.5
100	56.2	104.9	208.0

Buatlah plot data ini pada satu grafik tunggal yang diberi label dan judul, serta spesifikasi titik-titik datanya dengan gaya garis dan penanda yang berbeda-beda.

Solusi

Berikut adalah kode Python untuk menghasilkan grafik yang ditanyakan:

```python
import numpy as np
import matplotlib.pyplot as plt

# Suhu (degC) dan kelarutan (g/100 g air).
T = np.arange(0, 101, 20)
S_KCl = np.array([28.2, 34.2, 40.3, 45.6, 51.0, 56.2])
S_KBr = np.array([54.0, 65.9, 76.1, 85.9, 95.3, 104.9])
S_KI = np.array([127.8, 144.5, 161.0, 176.2, 191.5, 208.0])

plt.figure(figsize=(5,3))

def plot_solubilities():
    """Plot kelarutan halida sebagai fungsi dari suhu."""
    plt.plot(T, S_KCl, c='k', ls='-', lw=2, marker='s', ms=8,
     ↪  label='KCl')
    plt.plot(T, S_KBr, c='k', ls='--', lw=2, marker='^', ms=8,
     ↪  label='KBr')
    plt.plot(T, S_KI, c='k', ls=':', lw=2, marker='o', ms=8,
     ↪  label='KI')

    plt.xlabel(r'$T~(^{\circ}\text{C})$')
    plt.ylabel(r'$S~(\text{g}/100~\text{g}~\text{H}_2\text{O})$')
    plt.legend()

    # Tambahkan judul plot:
    ## untuk memastikannya muat, gunakan jeda baris (\n).
    plt.title('Ketergantungan kelarutan beberapa garam kalium\n
     ↪  terhadap suhu')

plot_solubilities()
plt.show()
```



Kasus 3 Visualisasi Distribusi Maxwell-Boltzmann

Distribusi Maxwell-Boltzmann adalah distribusi probabilitas memperoleh kelajuan, v, dari partikel-partikel dalam gas ideal untuk kesetimbangan termodinamika pada suhu, T:

$$f(v) = \left(\frac{m}{2\pi k_B T}\right)^{3/2} 4\pi v^2 \exp\left(-\frac{mv^2}{2k_B T}\right),$$

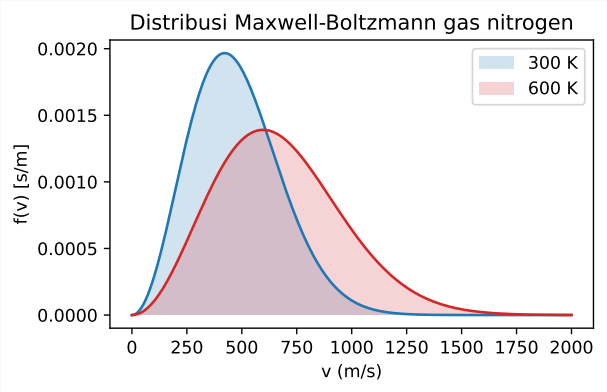
dengan m adalah massa partikel.
Buatlah fungsi Python untuk menghitung distribusi Maxwell-Boltzmann dan plot distribusinya untuk gas nitrogen (N_2) pada rentang $v = 0$ hingga 2000 m/s untuk dua nilai suhu berbeda: $T = 300$ K dan $T = 600$ K.

Solusi

Fungsi Python untuk menghitung distribusi ini sangat mudah ditulis dan digunakan:

```python
import numpy as np
import matplotlib.pyplot as plt

# Konstanta Boltzmann (J/K) dan 1 amu (kg)
kB, u = 1.381e-23, 1.661e-27

def fMB(v, T, m):
    """
    Memberikan nilai distribusi Maxwell-Boltzmann untuk molekul
    bermassa m (dalam kg) pada suhu T (dalam K), dengan kelajuan v
    ↪  (m/s).
    """
    fac = m / (2 * kB * T)
    return (fac / np.pi)**1.5 * 4 * np.pi * v**2 * np.exp(-fac * v**2)

# Grid kelajuan antara 0 dan vmax sebanyak 1000 titik
vmax = 2000
v = np.linspace(0, vmax, 1000)

# Massa molekuler (dalam kg) dari molekul N2.
mN2 = 2 * 14 * u

# Hitung distribusi Maxwell-Boltzmann untuk gas N2
# pada dua suhu yang berbeda (dalam K).
T1, T2 = 300, 600
f_N2_T1 = fMB(v, T1, mN2)
f_N2_T2 = fMB(v, T2, mN2)
```

Fungsi Matplotlib `fill_between` dapat digunakan untuk mengisi area di bawah sebuah kurva. Argumen dasarnya adalah `x`, yaitu koordinat-x dari area yang akan diisi, serta `y1` dan `y2` yang merupakan batas-batas dari koordinat-y. Jika tidak diberikan, nilai bawaan `y2=0` akan digunakan, yang berarti area di bawah kurva `y1(x)` hingga menyentuh sumbu-x akan diisi. Warna yang digunakan untuk mengisi area tersebut dapat ditetapkan dengan argumen `facecolor` (atau disingkat `fc`).

```python
plt.figure(figsize=(5,3))

def plot_MB():
    plt.plot(v, f_N2_T1)
    plt.plot(v, f_N2_T2, c='tab:red')

    plt.xlabel('v (m/s)')
    plt.ylabel('f(v) [s/m]')

    plt.fill_between(v, f_N2_T1, alpha=0.2, label=f'{T1} K')
    plt.fill_between(v, f_N2_T2, alpha=0.2, fc='tab:red',
                     label=f'{T2} K')

    plt.title('Distribusi Maxwell-Boltzmann gas nitrogen')
    plt.legend()

plot_MB()
plt.show()
```

Catatan: Argumen `alpha=0.2` pada kode di atas menetapkan tingkat opasitas/ketebalan isian warna sebesar 20%.